

HARE

Algorithm Specifications and Supporting Documentation (v1.5)

Tianrui Wang

Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China

Qicheng Teng

Institute for Advanced Study, Tsinghua University, Beijing, China

Jipeng Li

Xidian University, Xian, China

Anyu Wang

Institute for Advanced Study, Tsinghua University, Beijing, China

Jun Zhang

School of Mathematical Sciences, Capital Normal University, Beijing, China

Bo Pang

ICBC, Beijing, China

Chaojian Shen

ICBC, Beijing, China

Zhiyuan Qiu

Shandong Institute of BlockChain, Jinan, China

Chunhuan Zhao

Shield Lab, Huawei Technologies, Beijing, China

Sihuang Hu

Schooll of Cyber Science and Technology, Shandong University, Qingdao, China

Contact: Anyu Wang anyuwang@tsinghua.edu.cn

Table of Contents

1	Introduction	3
1.1	Properties of HARE	3
2	Preliminary	3
2.1	Notations	3
2.2	Coding Theory	4
2.3	Security Definitions	5
3	Specifications	6
3.1	Overview of HARE-PKE	6
3.2	Concatenated Reed-Muller and Reed-Solomon Codes	6
3.2.1	Duplicated Reed-Muller Codes	6
3.2.2	Reed-Solomon codes	7
3.3	Auxiliary Primitives	9
3.4	PKE	9
3.5	KEM	9
4	Parameters Choices	9
4.1	Parameter Sets	9
4.2	Ciphertext and key sizes	11
5	Basic Decoding Failure Rate Analysis	11
5.1	Analysis of Error Distribution	12
5.2	DFR of Reed-Muller Codes	12
5.3	DFR of Shortened Reed-Solomon Codes	13
6	Refined Decoding Failure Rate Analysis	14
6.1	Error Distribution Model	14
6.2	DFR of RM-RS codes	14
7	Security Analysis	15
7.1	Unbalanced-Quasi-Cyclic Syndrome Decoding Problem	15
7.2	Security Assumption	16
7.3	Security Proof	16
7.3.1	IND-CPA Security	16
7.3.2	IND-CCA2 Security	19
7.3.3	Known Attacks	19
8	Performance Analysis	20
8.1	Experimental Setup	20
8.2	Reference Implementation	20
8.3	Optimized Implementation	20

1 Introduction

The development of quantum algorithms such as Shor’s algorithm [1] has made the transition to post-quantum cryptography (PQC) inevitable. In 2016, the National Institute of Standards and Technology (NIST) initiated a process to standardize post-quantum public-key cryptography [2]. Code-based cryptography plays an important role in this effort, and in 2025, the Hamming Quasi-Cyclic (HQC) scheme [3] was selected as the first code-based key encapsulation mechanism (KEM) standard for PQC. While HQC offers a reliable decryption failure rate (DFR) estimation, its practical performance is hindered by larger public key and ciphertext sizes compared to lattice-based encryption schemes such as Kyber [4]. Consequently, optimizing the communication overhead (i.e., the public key and ciphertext sizes) of code-based schemes without compromising security and DFR has become a pressing research.

In this work, we present **HARE**, an optimized HQC-like encryption scheme that maintains provable security and DFR under the original HQC assumptions while achieving a substantial reduction in communication overhead. Compared to the original HQC, our parameter choices demonstrate that HARE reduces the combined size of the public key and ciphertext by 15.5%, 14.8%, and 15.2% for NIST security levels 1, 3, and 5, respectively.

Our main improvement comes from a refined decoding strategy that leverages *erasure error correction* during the decryption process. HQC employs a concatenated code structure combining Reed-Muller and Reed-Solomon codes to encode the message. The decoding procedure for this concatenated code proceeds in two stages: first, minimum-distance decoding is applied to the Reed-Muller code to correct bit flips in local blocks of the input noisy vector; second, the Reed-Solomon decoder treats each local block as a symbol and corrects any blocks that remain erroneous after the first stage, thereby recovering the original message.

In our approach, we observed that additional information can be extracted during the Reed-Muller decoding stage. Specifically, the distance between the input vector (restricted to a local block) and the nearest Reed-Muller codeword provides a useful hint about the reliability of the decoding result. If the decoding output has relatively low confidence, we erase that block and treat the corresponding position as an erasure during the subsequent Reed-Solomon decoding. We then employ an erasure-tolerant decoding algorithm for the Reed-Solomon code.

With this improved decoding strategy, we obtain a more effective decoding procedure that provably achieves a significantly lower DFR for a given set of code parameters. This allows us to reduce the length of the concatenated code while still meeting the DFR requirements for the same security level as HQC, leading to a reduction in the overall sizes of the public key and ciphertext.

1.1 Properties of HARE

The construction of HARE follows a framework that is similar to HQC, while introducing several improvements. In particular, HARE incorporates **four algorithmic modifications**: (i) a distance-informed erasure decoding strategy, (ii) an unbalanced error weight model, (iii) a ciphertext compression technique based on improved covering codes, and (iv) a refined DFR estimation model for tighter failure analysis.

These techniques do not affect the hardness assumptions, but instead focus on improving decoding efficiency and tightening the characterization of decoding failures. As a result, HARE achieves approximately **15%** reduction in overall communication overhead compared to HQC, while preserving the same security level.

2 Preliminary

2.1 Notations

Let $\mathbb{Z}$ denote the ring of integers, $\mathbb{F}_q$ denote the finite field with q elements. Let $\mathbb{F}_q^n$ denote the vector space, where each element is from $\mathbb{F}_q$. Let $n \in \mathbb{Z}$ be a primitive prime, i.e., 2 is a primitive root of n . Then we construct the polynomial ring $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$. Vectors are generally denoted by bold lowercase letters, e.g., $\mathbf{v}$, while matrices are denoted by uppercase letters, e.g., $\mathbf{G}$.

Unless otherwise specified, all vectors are treated as row vectors. Given $\mathbf{v} \in \mathbb{F}_2^n$, let $\text{rot}(\mathbf{v})$ denote the circulant matrix generated by $\mathbf{v}$, i.e.,

$$\text{rot}(\mathbf{v}) = \begin{pmatrix} v_0 & v_{n-1} & \cdots & v_1 \\ v_1 & v_0 & \cdots & v_2 \\ \vdots & \vdots & \ddots & \vdots \\ v_{n-1} & v_{n-2} & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}.$$

Since there is a natural one-to-one correspondence between the elements and operations in $\mathbb{F}_2^n$ and the polynomial ring $\mathcal{R}$, we will use vectors in $\mathbb{F}_2^n$ and polynomials in $\mathcal{R}$ interchangeably. Specifically, for $\mathbf{v} = (v_0, v_1, \dots, v_{n-1}) \in \mathbb{F}_2^n$, the corresponding element $\mathbf{v} = v_0 + v_1X + \cdots + v_{n-1}X^{n-1} \in \mathcal{R}$. The product of $\mathbf{u}, \mathbf{v} \in \mathcal{R}$ is $\mathbf{w} = \mathbf{u}\mathbf{v}$ with

$$w_k = \sum_{i+j \equiv k \pmod n} u_i v_j, \quad \forall k \in \{0, 1, \dots, n-1\},$$

which can be expressed as the product of $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$ with the operator $\text{rot}(\cdot)$ as:

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u} \times \text{rot}(\mathbf{v})^\top = (\text{rot}(\mathbf{u}) \times \mathbf{v}^\top)^\top = \mathbf{v} \times \text{rot}(\mathbf{u})^\top = \mathbf{v} \cdot \mathbf{u}.$$

Let $w_H(\mathbf{v})$ be the Hamming weight of the vector $\mathbf{v}$, i.e., the number of nonzero positions of $\mathbf{v}$. Let $\mathcal{R}_w := \{\mathbf{v} \in \mathcal{R} | w_H(\mathbf{v}) = w\}$. For $\mathbf{v} = (v_0, \dots, v_{n-1}) \in \mathbb{F}_q^n$, we define $\text{Truncate}(\mathbf{v}, l) := (v_0, \dots, v_{l-1})$ as the function that truncate the vector $\mathbf{v}$ and only keep the l least-significant positions.

2.2 Coding Theory

Definition 1 (Linear Code). A q -ary linear code $\mathcal{C}$ is a linear subspace of $\mathbb{F}_q^n$. If $\mathcal{C}$ has dimension k , it is called a $[n, k]$ or $[n, k]_q$ code. The code $\mathcal{C}$ can be equivalently described in the following ways:

- (i) $\mathcal{C} = \{\mathbf{m}\mathbf{G} \mid \mathbf{m} \in \mathbb{F}_q^k\}$, where $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ is called a generator matrix.
- (ii) $\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_q^n \mid \mathbf{H}\mathbf{c} = \mathbf{0}\}$, where $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ is called a parity check matrix.

Definition 2 (Minimum Distance). The minimum distance of a code $\mathcal{C}$ is

$$\min\{d(\mathbf{x}, \mathbf{y}) \mid \mathbf{x} \in \mathcal{C}, \mathbf{y} \in \mathcal{C}, \mathbf{x} \neq \mathbf{y}\}$$

If the minimum distance of $\mathcal{C}$ is d , then we also describe it as a $[n, k, d]$ code.

Definition 3 (Hamming Codes). A Hamming code over $\mathbb{F}_2$ has parameters $[2^m - 1, 2^m - m - 1, 3]$ for any integer $m \geq 2$.

Definition 4 (Concatenated Codes). A concatenated code over $\mathbb{F}_2$ consists of an external code $\mathcal{C}_e$ with parameters $[n_e, k_e, d_e]$ over $\mathbb{F}_q$ and an internal code $\mathcal{C}_i$ with parameters $[n_i, k_i, d_i]$ over $\mathbb{F}_2$, where $q = 2^{k_i}$. The resulting concatenated code is a binary code with parameters $[N = n_e n_i, K = k_e k_i, D \geq d_e d_i]$.

Definition 5 (Covering Code). A linear $[n, k]_q$ code $\mathcal{C}$ has covering radius R if any of the following equivalent conditions holds:

- (i) The value R is the smallest integer such that the space $\mathbb{F}_q^n$ is covered by Hamming balls of radius R centered at the codewords of $\mathcal{C}$.
- (ii) Every vector in $\mathbb{F}_q^{n-k}$ can be expressed as a linear combination of at most R columns of the parity check matrix of $\mathcal{C}$, and R is the smallest value with this property.

The encoding of covering codes is obtained by multiplying the generator matrix. When the covering radius R is small, we can decode any covering codes with the table-based syndrome decoding technique similar in [5].

With the algorithm in Algorithm 1, decoding is guaranteed to succeed when a covering code is employed. Regarding efficiency, the table $\mathcal{T}$ can be precomputed offline in $\sum_{i=0}^R \binom{n}{i}$ operations by enumerating all error vectors of weight at most R . During the online phase, the decoding complexity is dominated by the computation of $\mathbf{H}\mathbf{v}^T$, which requires $O(n(n-k))$ operations.

Algorithm 1 Syndrome Table Decoding**Input:** Parity check matrix $\mathbf{H}$, received vector $\mathbf{v}$ **Parameter:** Covering radius R **Output:** Error vector $\mathbf{e}$

```

1: Initialize an empty table  $\mathcal{T}$ 
2: for all  $\mathbf{e} \in \mathbb{F}_2^n$  with  $w_H(\mathbf{e}) \leq R$  do
3:    $\mathbf{s} \leftarrow \mathbf{H}\mathbf{e}^T$ 
4:   if  $\mathbf{s} \notin \mathcal{T}$  or  $w_H(\mathbf{e}) < w_H(\mathcal{T}[\mathbf{s}])$  then
5:      $\mathcal{T}[\mathbf{s}] \leftarrow \mathbf{e}$ 
6:   end if
7: end for
8:  $\mathbf{s}_v \leftarrow \mathbf{H}\mathbf{v}^T$ 
9:  $\mathbf{e} \leftarrow \mathcal{T}[\mathbf{s}_v]$ 
10: return  $\mathbf{e}$ 

```

Definition 6 (Kaikkonen-Rosendahl Code). Let $\mathcal{C}_{KR}$ be the binary $[51, 41]_2$ code with the parity check matrix $\mathbf{H}_{KR} = [\mathbf{I}_{10} \mathbf{M}_{KR}]$, where $\mathbf{M}_{KR}$ is the 10×41 binary matrix with the following columns in hexadecimal notation:

$$\begin{aligned} \mathbf{M}_{KR} = [& 1B6, 193, 1CC, 187, 1F6, F7, 16E, 140, 3C, 296, 22F, 303, 381, 365, \\ & 11D, 1A3, 274, 2F2, 254, 56, F, 41, 357, 208, 34, 329, 28D, 31D, 3D5, \\ & 129, 3D7, B7, 3EC, 2E2, 23C, AD, 34E, 155, 2E6, 371, D4] \end{aligned} \quad (1)$$

It is proven in [6] that $\mathcal{C}_{KR}$ is a $[51, 41]_2$ code with covering radius 2. In our scheme, we choose $\mathcal{C}_2$ as the direct product of $\mathcal{C}_{KR}$. The direct product code $\mathcal{C}_2$ is decoded in a block-wise manner, where the decoding of $\mathcal{C}_{KR}$ follows the procedure described in Algorithm 1.

2.3 Security Definitions

We introduce the security notions IND-CPA and IND-CCA2 for PKE and KEM, respectively. The corresponding security games and advantages are defined below.

Definition 7. Let $\text{PKE} = (\text{Keygen}, \text{Encrypt}, \text{Decrypt})$ be a public-key encryption scheme, the IND-CPA advantage of a probabilistic polynomial time adversary $\mathcal{A} = (\mathcal{A}_{\text{CHOOSE}}, \mathcal{A}_{\text{GUESS}})$ is defined as:

$$\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = |\Pr[\text{PKE.IND-CPA}(\mathcal{A}) = 1] - \frac{1}{2}|,$$

where the IND-CPA game is defined as follows.

(IND-CPA Game)

1. $(\mathbf{ek}, \mathbf{dk}) \leftarrow \text{Keygen}()$
2. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\mathbf{ek})$
3. $b \xleftarrow{\$} \{0, 1\}$, $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$
4. $\mathbf{c}_{\text{PKE}} \leftarrow \text{Encrypt}(\mathbf{ek}, \mathbf{m}_b, \theta)$
5. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\mathbf{ek}, \mathbf{c}_{\text{PKE}})$
6. **return** $(b = b')$

Definition 8. Let $\text{KEM} = (\text{Keygen}, \text{Encaps}, \text{Decaps})$ be a key encapsulation mechanism, the IND-CCA2 advantage of a probabilistic polynomial time adversary $\mathcal{A} = \mathcal{A}_{\text{GUESS}}$ with access to a decapsulation oracle Decaps is defined as:

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) = |\Pr[\text{KEM.IND-CCA2}(\mathcal{A}) = 1] - \frac{1}{2}|,$$

where the IND-CCA2 game is defined as follows.

(IND-CCA2 Game)

1. $(\mathbf{ek}, \mathbf{dk}) \leftarrow \text{Keygen}()$

2. $b \xleftarrow{\$} \{0, 1\}$
3. $(\mathbf{K}_0, \mathbf{c}_{\text{KEM}}) \leftarrow \text{Encaps}(\mathbf{ek})$
4. $\mathbf{K}_1 \xleftarrow{\$} \mathcal{B}^{|\mathbf{K}_1|}$
5. $b' \leftarrow \mathcal{A}_{\text{GUESS}}^{\text{DECAPS}(\cdot)}(\mathbf{K}_b, \mathbf{c}_{\text{KEM}})$ where the adversary is not allowed to query $\mathbf{c}_{\text{KEM}}$ to the decapsulation oracle
6. return $(b = b')$

3 Specifications

3.1 Overview of HARE-PKE

Before presenting the formal definition of HARE-PKE, we first describe the main idea and the procedures of the scheme at a high level. Compared to the original HQC scheme [3], in addition to the distance-informed erasure decoding as described in Section 1, we introduce asymmetric error weights for $\mathbf{r}_1, \mathbf{r}_2$, and $\mathbf{e}$ and accordingly adapt the underlying security assumptions. This design is inspired by the use of asymmetric error distributions in lattice-based constructions [7]. Moreover, we incorporate the ciphertext compression technique proposed in [8] with a better covering code.

- **Keygen**($\cdot$): samples $\mathbf{h} \xleftarrow{\$} \mathcal{R}$, $\mathbf{x}, \mathbf{y} \xleftarrow{\$} \mathcal{R}_w \times \mathcal{R}_w$, sets $\mathbf{sk} = (\mathbf{x}, \mathbf{y})$ and $\mathbf{pk} = (\mathbf{h}, \mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$, and returns $(\mathbf{pk}, \mathbf{sk})$
- **Encrypt**($\mathbf{pk}, \mathbf{m}$): generates $\mathbf{e} \xleftarrow{\$} \mathcal{R}_{w_e}$, $\mathbf{r} = (\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\$} \mathcal{R}_{w_r} \times \mathcal{R}_{w_r}$, sets $\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$ and $\mathbf{v} = \text{Truncate}(\mathbf{m}\mathbf{G}_1 + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, l_1)$, splits $\mathbf{v}_1 = \mathbf{v}[0, l_2 - 1]$ and $\mathbf{v}_2 = \mathbf{v}[l_2, l_1]$, decodes $\tilde{\mathbf{m}} = \mathcal{C}_2.\text{Decode}(\mathbf{v}_1)$, returns $\mathbf{c} = (\mathbf{u}, \tilde{\mathbf{m}}, \mathbf{v}_2)$
- **Decrypt**($\mathbf{sk}, \mathbf{c}$): recovers $\tilde{\mathbf{v}} = (\tilde{\mathbf{m}}\mathbf{G}_2 \parallel \mathbf{v}_2)$, returns $\mathcal{C}_1.\text{Decode}(\tilde{\mathbf{v}} - \text{Truncate}(\mathbf{u} \cdot \mathbf{y}, l_1))$

The matrices $\mathbf{G}_1$ and $\mathbf{G}_2$ denote the generator matrices of the codes $\mathcal{C}_1$ and $\mathcal{C}_2$, respectively. In our scheme, we choose concatenated Reed-Muller codes of length n_2 and Reed-Solomon codes of length n_1 to construct $\mathcal{C}_1$, and the KR code $[51, 41]_2$ for $\mathcal{C}_2$. The function $\text{Truncate}(\mathbf{v}, l)$ splits the vector $\mathbf{v}$ into two segments and keeps only the first segment of length l .

Correctness. Let $\tilde{\mathbf{e}} = \tilde{\mathbf{m}}\mathbf{G}_2 - \mathbf{v}_1$ denote the noise introduced in $\mathcal{C}_2$. For simplicity, we omit truncation and assume all vectors have matching lengths. The decryption can then be written as

$$\begin{aligned}
 \tilde{\mathbf{v}} - \mathbf{u}\mathbf{y} &= (\tilde{\mathbf{m}}\mathbf{G}_2 \parallel \mathbf{v}_2) - \mathbf{u}\mathbf{y} \\
 &= (\mathbf{v}_1 + \tilde{\mathbf{e}} \parallel \mathbf{v}_2) - \mathbf{u}\mathbf{y} \\
 &= \mathbf{v} + \tilde{\mathbf{e}} - \mathbf{u}\mathbf{y} \\
 &= \mathbf{m}\mathbf{G}_1 + \mathbf{s}\mathbf{r}_2 + \mathbf{e} + \tilde{\mathbf{e}} - (\mathbf{r}_1 + \mathbf{h}\mathbf{r}_2)\mathbf{y} \\
 &= \mathbf{m}\mathbf{G}_1 + \mathbf{x}\mathbf{r}_2 - \mathbf{y}\mathbf{r}_1 + \mathbf{e} + \tilde{\mathbf{e}}.
 \end{aligned}$$

Since $\mathbf{x}, \mathbf{y}, \mathbf{r}_1, \mathbf{r}_2, \mathbf{e}, \tilde{\mathbf{e}}$ all have low Hamming weight, the equivalent error in $\mathcal{C}_1$ is $\mathbf{x}\mathbf{r}_2 - \mathbf{y}\mathbf{r}_1 + \mathbf{e} + \tilde{\mathbf{e}}$ with relatively low weight. Decryption succeeds if and only if this equivalent error is within the error-correcting capability of $\mathcal{C}_1$.

3.2 Concatenated Reed-Muller and Reed-Solomon Codes

In this section, we introduce how we construct $\mathcal{C}_1$ with concatenated RM and RS codes and our erasure decoding algorithm.

3.2.1 Duplicated Reed-Muller Codes

We propose a modified decoding algorithm, $\text{Decode}'(\cdot)$, for duplicated Reed-Muller codes, as described in Algorithm 2. The algorithm outputs either a message $\mathbf{m}$ or an erasure symbol $\perp$, depending on the confidence of the decoding result.

The main modification compared to the classical decoding in [3] is as follows. Instead of directly decoding to the nearest codeword, we first identify the two codewords with the smallest Hamming distances to the received vector. If the difference between these two distances exceeds a threshold α ,

the vector is decoded to the nearest codeword. Otherwise, the decoder outputs an erasure symbol $\perp$, indicating insufficient confidence. This approach reduces the probability of incorrect decoding by explicitly marking uncertain positions as erasures, which can then be handled in the subsequent Reed-Solomon decoding stage.

For context, a Reed-Muller code $\text{RM}(r, m)$ with positive integer m and $0 \leq r \leq m$ has parameters $[n, k, d_{\min}] = [2^m, \sum_{i=0}^r \binom{m}{i}, 2^{m-r}]$, and duplicated RM codes are constructed by repeating each codeword multiple times [9]. Encoding is performed by multiplying the message with the generator matrix.

Algorithm 2 Decode' for RM

Input: Vector $\mathbf{x}$ to be corrected

Parameter: RM code of $[n_i, k_i, d_i]$ over $\mathbb{F}_2$ with generation matrix $\mathbf{G}$, threshold α

Output: Message $\mathbf{m}$ or $\perp$

```

1:  $\mathbf{m}_1, \mathbf{m}_2 \leftarrow \mathbf{0}, \mathbf{0}$ 
2: Construct  $\mathcal{D} \leftarrow \{(d, \mathbf{m}) | d = w_H(\mathbf{m}\mathbf{G} - \mathbf{x})\}$  with Hadamard transform
3: Sort  $\mathcal{D}$  in ascending order of  $d$ 
4:  $(d_1, \mathbf{m}_1), (d_2, \mathbf{m}_2)$  be the first two elements in  $\mathcal{D}$ 
5: if  $d_2 - d_1 > \alpha$  then
6:   return  $\mathbf{m}_1$ 
7: else
8:   return  $\perp$ 
9: end if
```

3.2.2 Reed-Solomon codes

When performing the decoding function of shortened Reed-Solomon codes, we employ a modified decoding procedure that tolerates erasures. We summarize the encoding and decoding procedures of the shortened Reed-Solomon code as follows.

Let p be a prime number and q be a power of p . A Reed-Solomon code $\text{RS}[n, k, d_{\min}]$ with symbols in $\mathbb{F}_q$ has block length $n = q - 1$, dimension k , and minimum distance $d_{\min}$. A codeword with g errors and h erasures can be decoded correctly if $2g + h \leq n - k$.

Shortened Reed-Solomon codes. For the three HARE parameter sets, we construct shortened Reed-Solomon codes from classical RS codes with $k = 16, 24$, and 32 . The resulting code parameters are summarized in Table 1. Note that shortening a Reed-Solomon code does not affect its error-correcting capacity.

Code	n	k	$d_{\min}$
RS-S1	40	16	25
RS-S3	50	24	27
RS-S5	70	32	39

Table 1: The parameter choices for shortened RS codes in HARE.

Constructing and encoding shortened Reed-Solomon codes. For HARE, we work over $\mathbb{F}_{2^m}$ with $m = 8$. To construct this field, we use the primitive polynomial $1 + \alpha^2 + \alpha^3 + \alpha^4 + \alpha^8$ of degree 8 with a primitive element α . The generator polynomials of RS-S1, RS-S3, and RS-S5 can be computed as $g(x) = \prod_{i=1}^{n-k} (x + \alpha^i)$.

The encoding of shortened Reed-Solomon code is the same as that of classical RS code. We denote the message $u = (u_0, \dots, u_{k-1})$ by the polynomial $u(x) = u_0 + \dots + u_{k-1}x^{k-1}$. Following [10], the codeword polynomial $c(x)$ is given by $c(x) = b(x) + x^{n-k}u(x)$, where $b(x)$ is the remainder of the division of the polynomial $x^{n-k}u(x)$ by $g(x)$.

Decoding shortened Reed-Solomon codes. In the decoding procedure, we should decode Reed-Solomon codes with both errors and erasures. We denote the received polynomial $r(x) = r_0 + \dots + r_{n-1}x^{n-1}$. Suppose there are g errors at positions $j_1, \dots, j_g$ and h erasures at positions $l_1, \dots, l_h$, then the error polynomial and the erasure polynomial can be denoted as $e(x) = e_{j_1}x^{j_1} + \dots + e_{j_g}x^{j_g}$ and $w(x) = w_{l_1}x^{l_1} + \dots + w_{l_h}x^{l_h}$. For simplicity, we define location numbers $\beta_i = \alpha^{j_i}$ and $\eta_t = \alpha^{l_t}$ to indicate the locations of errors and erasures. The received polynomial can be represented by $r(x) = v(x) + e(x) + w(x)$, where $v(x)$ is the original codeword.

To describe the locations of errors and erasures, we define the error locator polynomial $\sigma(x)$, the erasure locator polynomial $\gamma(x)$ and the total locator polynomial $\Lambda(x)$ as follows.

$$\begin{aligned}\sigma(x) &= \prod_{i=1}^g (1 + \beta_i x) = 1 + \sigma_1 x + \dots + \sigma_g x^g, \\ \gamma(x) &= \prod_{t=1}^h (1 + \eta_t x) = 1 + \gamma_1 x + \dots + \gamma_h x^h, \\ \Lambda(x) &= \sigma(x)\gamma(x)\end{aligned}$$

We then define the syndromes $S_1, \dots, S_{n-k}$ as $S_i = r(\alpha^i)$. We have $S_i = e(\alpha^i) + w(\alpha^i)$ since $v(\alpha^i) = 0$. With the erasure locations known, we obtain $n - k$ equations with $2g + h$ unknowns including e_{j_i}, β_i and w_{l_t} .

$$\begin{cases} S_1 = \sum_{i=1}^g e_{j_i} \beta_i + \sum_{t=1}^h w_{l_t} \eta_t \\ S_2 = \sum_{i=1}^g e_{j_i} \beta_i^2 + \sum_{t=1}^h w_{l_t} \eta_t^2 \\ \dots \\ S_{n-k} = \sum_{i=1}^g e_{j_i} \beta_i^{n-k} + \sum_{t=1}^h w_{l_t} \eta_t^{n-k} \end{cases} \quad (2)$$

Our goal is to solve the $2g + h$ unknowns in (2). According to [10], the decoding procedure can be summarized in the following steps:

1. Compute the syndromes $S_1, \dots, S_{n-k}$ by evaluating $r(\alpha^i)$ for each i .
2. Compute the modified syndrome polynomial $T(x) = S(x)\gamma(x) \bmod x^{n-k}$ **with the erasures considered**, where $S(x) = 1 + S_1 x + \dots + S_{n-k} x^{n-k}$ is the syndrome polynomial. Then use Berlekamp-Massey (BM) to compute the error locator polynomial $\sigma(x)$.
3. Find the error location numbers by calculating the roots of the polynomial $\sigma(x)$ and returning their inverses. This step can be implemented through an additive Fast Fourier Transform algorithm.
4. Compute the polynomial $Z(x) = S(x)\Lambda(x) \bmod x^{n-k}$. This polynomial is used to determine the error and erasure values.
5. Compute the error and erasure values at position s by

$$\varepsilon_s = \frac{\alpha^s Z(\alpha^{-s})}{Z'(\alpha^{-s})} \quad (3)$$

6. Correct errors and restore erasures, i.e., $v(x) = r(x) + e(x) + w(x)$.

The main modification of the decoding procedure with erasures from the classical one in [3] is step 2. In order to take the erasure positions into consideration when processing the syndromes, we calculate the modified syndrome polynomial $T(x) = S(x)\gamma(x) \bmod x^{n-k} = 1 + T_1 x + \dots + T_{n-k-1} x^{n-k-1}$, where $T_1, \dots, T_{n-k-1}$ are called the modified syndromes. Following [10], the modified syndromes and the coefficients $\sigma_1, \dots, \sigma_g$ satisfy

$$T_i + \sigma_1 T_{i-1} + \dots + \sigma_g T_{i-g} = 0, \quad i = g+1, g+2, \dots, n-k-h, \quad (4)$$

which is a linear feedback shift register with sequence $T_1, \dots, T_{n-k-h}$ and coefficients $\sigma_1, \dots, \sigma_g$, thus we can compute the coefficients by BM algorithm [10]. Notice that the relation holds only for the first $n - k - h$ modified syndromes, so the BM procedure contains only $n - k - h$ loops of iteration rather than $n - k$.

After retrieving the coefficients of $\sigma(x)$, we can compute all the error positions by finding the inverses of the roots of $\sigma(x)$. From then on, errors and erasures can be treated equivalently, since all the positions are clear. The following steps are the same as those in [3].

3.3 Auxiliary Primitives

Prior to discussing the instantiation of HARE-PKE and HARE-KEM, we introduce several auxiliary primitives used for pseudorandom byte generation, hashing, and vector sampling.

- **Extendable Output Function:** XOF generates a pseudorandom byte string of arbitrary length from a given short seed.
- **Hash Functions:** G, H, I, J denote cryptographic hash functions.
- **Vector Sampling:** `SampleVec` and `SampleFixedWeightVec` are employed to sample random vectors from the spaces $\mathcal{R}$ and $\mathcal{R}_w$, respectively.

We omit a detailed discussion of these primitives here. Our implementation follows the programming interface provided by Institute of Commercial Cryptography Standards (ICCS) [11], which defines standardized APIs for public-key cryptographic algorithms, including key generation, encapsulation, and decapsulation.

3.4 PKE

We now describe the HARE-PKE scheme. The formal procedures are given in [Algorithm 3](#), [Algorithm 4](#), and [Algorithm 5](#).

Algorithm 3 PKE.Keygen(seed_{PKE})

Input: Seed seed_{PKE}

Output: Encryption key ek_{PKE} and decryption key dk_{PKE}

```

1:  $\text{seed}_{\text{PKE.dk}}, \text{seed}_{\text{PKE.ek}} \leftarrow I(\text{seed}_{\text{PKE}})$ 
2:  $\text{ctx}_{\text{PKE.dk}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{PKE.dk}})$ 
3:  $\text{ctx}_{\text{PKE.ek}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{PKE.ek}})$ 
4:  $\text{ctx}_{\text{PKE.dk}}, \mathbf{y} \leftarrow \text{SampleFixedWeightVec}(\text{ctx}_{\text{PKE.dk}}, \mathcal{R}_w)$ 
5:  $\text{ctx}_{\text{PKE.dk}}, \mathbf{x} \leftarrow \text{SampleFixedWeightVec}(\text{ctx}_{\text{PKE.dk}}, \mathcal{R}_w)$ 
6:  $\text{dk}_{\text{PKE}} \leftarrow \text{seed}_{\text{PKE.dk}}$ 
7:  $\text{ctx}_{\text{PKE.ek}}, \mathbf{h} \leftarrow \text{SampleVec}(\text{ctx}_{\text{PKE.ek}}, \mathcal{R})$ 
8:  $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$ 
9:  $\text{ek}_{\text{PKE}} \leftarrow (\text{seed}_{\text{PKE.ek}}, \mathbf{s})$ 
10: return  $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}})$ 

```

3.5 KEM

We then construct HARE-KEM from HARE-PKE using the Fujisaki-Okamoto transformation [12] and the HHK framework with implicit rejection [13]. The formal procedures are specified in [Algorithm 6](#), [Algorithm 7](#), and [Algorithm 8](#).

4 Parameters Choices

4.1 Parameter Sets

We design the parameter choices in [Table 2](#), which includes the classic security, quant security, length of the quasi-cyclic code n and dimension of the concatenated code k . We design six parameter sets to facilitate comparison with international benchmarks. HARE-1, HARE-3, and HARE-5 are aligned with the NIST security levels 1, 3, and 5, respectively. In addition, HARE-2, HARE-5, HARE-7, and HARE-9 are selected to meet the security requirements specified in this call.

Algorithm 4 $\text{PKE.Encrypt}(\text{ek}_{\text{PKE}}, \mathbf{m}, \theta)$ **Input:** Encryption key ek_{PKE} , message $\mathbf{m}$ and randomness θ **Output:** Ciphertext $\mathbf{c}$

```

1:  $\text{seed}_{\text{PKE.ek}} \leftarrow \text{ek}_{\text{PKE}}[0 : |\text{seed}| - 1]$ 
2:  $\text{ctx}_{\text{PKE.ek}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{PKE.ek}})$ 
3:  $\text{ctx}_{\text{PKE.ek}}, \mathbf{h} \leftarrow \text{SampleVec}(\text{ctx}_{\text{PKE.ek}}, \mathcal{R})$ 
4:  $\mathbf{s} \leftarrow \text{ek}_{\text{PKE}}[|\text{seed}| : |\text{seed}| + |\mathbf{s}| - 1]$ 
5:  $\text{ctx}_{\theta} \leftarrow \text{XOF.Init}(\theta)$ 
6:  $\mathbf{e} \leftarrow \text{SampleFixedWeightVec}(\text{ctx}_{\theta}, \mathcal{R}_{w_e})$ 
7:  $\mathbf{r}_1 \leftarrow \text{SampleFixedWeightVec}(\text{ctx}_{\theta}, \mathcal{R}_{w_r})$ 
8:  $\mathbf{r}_2 \leftarrow \text{SampleFixedWeightVec}(\text{ctx}_{\theta}, \mathcal{R}_{w_r})$ 
9:  $\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$ 
10:  $\mathbf{v} \leftarrow \mathbf{m}\mathbf{G}_1 + \text{Truncate}(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, l_1)$ 
11:  $\mathbf{v}_1 \leftarrow \mathbf{v}[0, l_2 - 1]$ 
12:  $\mathbf{v}_2 \leftarrow \mathbf{v}[l_2, l_1 - 1]$ 
13:  $\tilde{\mathbf{m}} \leftarrow \mathcal{C}_2.\text{Decode}(\mathbf{v}_1)$ 
14:  $\mathbf{c} \leftarrow (\mathbf{u}, \tilde{\mathbf{m}}, \mathbf{v}_2)$ 
15: return  $\mathbf{c}$ 

```

Algorithm 5 $\text{PKE.Decrypt}(\text{dk}_{\text{PKE}}, \mathbf{c})$ **Input:** Decryption key dk_{PKE} and ciphertext $\mathbf{c}$ **Output:** Message $\mathbf{m}$

```

1:  $\text{ctx}_{\text{PKE.dk}} \leftarrow \text{XOF.Init}(\text{dk}_{\text{PKE}})$ 
2:  $\text{ctx}_{\text{PKE.dk}}, \mathbf{y} \leftarrow \text{SampleFixedWeightVec}(\text{ctx}_{\text{PKE.dk}}, \mathcal{R}_w)$ 
3:  $\mathbf{u} \leftarrow \mathbf{c}[0 : |\mathbf{u}|]$ 
4:  $\tilde{\mathbf{m}} \leftarrow \mathbf{c}[|\mathbf{u}| : |\mathbf{u}| + |\tilde{\mathbf{m}}| - 1]$ 
5:  $\mathbf{v}_2 \leftarrow \mathbf{c}[|\mathbf{u}| + |\tilde{\mathbf{m}}| : |\mathbf{c}| - 1]$ 
6:  $\tilde{\mathbf{v}} \leftarrow (\tilde{\mathbf{m}}\mathbf{G}_2, \mathbf{v}_2)$ 
7:  $\mathbf{m} \leftarrow \mathcal{C}_1.\text{Decode}(\tilde{\mathbf{v}} - \text{Truncate}(\mathbf{u}\mathbf{y}, l_1))$ 
8: return  $\mathbf{m}$ 

```

Algorithm 6 $\text{HARE-KEM.Keygen}()$ **Input:** None**Output:** Encapsulation key ek_{KEM} and decapsulation key dk_{KEM}

```

1:  $\text{seed}_{\text{KEM}} \xleftarrow{\$} \mathcal{B}^{|\text{seed}|}$ 
2:  $\text{ctx}_{\text{KEM}} \leftarrow \text{XOF.Init}(\text{seed}_{\text{KEM}})$ 
3:  $(\text{ctx}_{\text{KEM}}, \text{seed}_{\text{PKE}}) \leftarrow \text{XOF.GetBytes}(\text{ctx}_{\text{KEM}}, |\text{seed}|)$ 
4:  $(\text{ctx}_{\text{KEM}}, \sigma) \leftarrow \text{XOF.GetBytes}(\text{ctx}_{\text{KEM}}, |\mathbf{K}|)$ 
5:  $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}}) \leftarrow \text{HARE-PKE.Keygen}(\text{seed}_{\text{PKE}})$ 
6:  $\text{ek}_{\text{KEM}} \leftarrow \text{ek}_{\text{PKE}}$ 
7:  $\text{dk}_{\text{KEM}} \leftarrow (\text{ek}_{\text{KEM}}, \text{dk}_{\text{PKE}}, \sigma, \text{seed}_{\text{KEM}})$ 
8: return  $(\text{ek}_{\text{KEM}}, \text{dk}_{\text{KEM}})$ 

```

Algorithm 7 $\text{HARE-KEM.Encaps}(\text{ek}_{\text{KEM}})$ **Input:** Encapsulation key ek_{KEM} **Output:** Shared secret key $\mathbf{K}$ and ciphertext $\mathbf{c}_{\text{KEM}}$

```

1:  $\mathbf{m} \xleftarrow{\$} \mathcal{B}^{|\mathbf{K}|}$ 
2:  $\text{salt} \xleftarrow{\$} \mathcal{B}^{|\text{salt}|}$ 
3:  $(K, \theta) \leftarrow \text{G}(\text{H}(\text{ek}_{\text{KEM}}) \parallel \mathbf{m} \parallel \text{salt})$ 
4:  $\mathbf{c}_{\text{PKE}} \leftarrow \text{HARE-PKE.Encrypt}(\text{ek}_{\text{KEM}}, \mathbf{m}, \theta)$ 
5:  $\mathbf{c}_{\text{KEM}} \leftarrow (\mathbf{c}_{\text{PKE}}, \text{salt})$ 
6: return  $(\mathbf{K}, \mathbf{c}_{\text{KEM}})$ 

```

Algorithm 8 HARE-KEM.Decaps($\mathbf{dk}_{\text{KEM}}, \mathbf{c}_{\text{KEM}}$)**Input:** Decapsulation key $\mathbf{dk}_{\text{KEM}}$ and ciphertext $\mathbf{c}_{\text{KEM}}$ **Output:** Shared secret key $\mathbf{K}'$

```

1:  $\mathbf{ek}_{\text{KEM}} \leftarrow \mathbf{dk}_{\text{KEM}}[0 : |\mathbf{ek}_{\text{KEM}}|]$ 
2:  $\mathbf{dk}_{\text{PKE}} \leftarrow \mathbf{dk}_{\text{KEM}}[|\mathbf{ek}_{\text{KEM}}| : |\mathbf{ek}_{\text{KEM}}| + |\mathbf{dk}_{\text{PKE}}|]$ 
3:  $\sigma \leftarrow \mathbf{dk}_{\text{KEM}}[|\mathbf{ek}_{\text{KEM}}| + |\mathbf{dk}_{\text{PKE}}| : |\mathbf{ek}_{\text{KEM}}| + |\mathbf{dk}_{\text{PKE}}| + |\sigma|]$ 
4:  $\mathbf{c}_{\text{PKE}} \leftarrow \mathbf{c}_{\text{KEM}}[0 : |\mathbf{c}_{\text{PKE}}|]$ 
5:  $\mathbf{salt} \leftarrow \mathbf{c}_{\text{KEM}}[|\mathbf{c}_{\text{PKE}}| : |\mathbf{c}_{\text{PKE}}| + |\mathbf{salt}|]$ 
6:  $\mathbf{m}' \leftarrow \text{HARE-PKE.Decrypt}(\mathbf{dk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$ 
7:  $(\mathbf{K}', \theta') \leftarrow \mathcal{G}(\mathcal{H}(\mathbf{ek}_{\text{KEM}}) \parallel \mathbf{m}' \parallel \mathbf{salt})$ 
8:  $\mathbf{c}'_{\text{PKE}} \leftarrow \text{HARE-PKE.Encrypt}(\mathbf{ek}_{\text{KEM}}, \mathbf{m}', \theta')$ 
9:  $\mathbf{c}'_{\text{KEM}} \leftarrow (\mathbf{c}'_{\text{PKE}}, \mathbf{salt})$ 
10:  $\bar{\mathbf{K}} \leftarrow \mathcal{J}(\mathcal{H}(\mathbf{ek}_{\text{KEM}}) \parallel \sigma \parallel \mathbf{c}_{\text{KEM}})$ 
11: if  $\mathbf{m}' = \perp$  or if  $\mathbf{c}'_{\text{KEM}} \neq \mathbf{c}_{\text{KEM}}$  then
12:    $\mathbf{K}' \leftarrow \bar{\mathbf{K}}$ 
13: end if
14: return  $\mathbf{K}'$ 

```

Instance	Classical Security	Quantum Security	n	k	$w = w_r$	w_e	n_1	n_2	α	DFR
HARE-1	128	64	16067	128	66	105	41	384	7	$< 2^{-128}$ (Model 2)
HARE-2	128	80	20899	128	79	145	32	640	11	$< 2^{-130}$ (Model 2)
HARE-3	192	96	32653	192	100	163	51	640	11	$< 2^{-194}$ (Model 2)
HARE-5	256	128	52379	256	131	224	81	640	9	$< 2^{-258}$ (Model 2)
HARE-7	384	192	104869	384	193	361	91	1152	15	$< 2^{-384}$ (Model 1)
HARE-9	512	256	173981	512	259	449	151	1152	13	$< 2^{-519}$ (Model 1)

Table 2: The parameter choices for HARE.

The parameters w, w_r and w_e represent the Hamming weight of vectors $(\mathbf{x}, \mathbf{y})$, $(\mathbf{r}_1, \mathbf{r}_2)$ and $\mathbf{e}$, respectively. For code choices, n_1 denotes the length of RS code, n_2 denotes the length of RM code and α denotes the threshold in the decoding procedure. We upper bound the DFR for each parameter set using either Model 1 in Section 5 or Model 2 in Section 6. Due to the computational cost of Model 2, we apply it only to HARE-1/2/3/5. For HARE-7/9, we instead use the more conservative upper bound obtained from Model 1.

4.2 Ciphertext and key sizes

The encapsulation key $\mathbf{ek}_{\text{KEM}}$ has size $|\mathbf{ek}_{\text{KEM}}| = |\mathbf{seed}| + \lceil n/8 \rceil$ while the decapsulation key $\mathbf{dk}_{\text{KEM}}$ has size $|\mathbf{dk}_{\text{KEM}}| = |\mathbf{ek}_{\text{KEM}}| + |\mathbf{seed}| + \lceil \sigma \rceil + |\mathbf{seed}|$ with the compressed format used. The ciphertext $\mathbf{c}_{\text{KEM}}$ has size $|\mathbf{c}_{\text{KEM}}| = \lceil n/8 \rceil + \lceil (l_1 - l_2 + k_2 \lceil l_1/n_2 \rceil)/8 \rceil + |\mathbf{salt}|$ while the shared key $\mathbf{K}$ has size $|\mathbf{K}|$. In HARE, the sizes of the seed, salt, and shared key are aligned with the classical security level. In HQC, the sizes are $|\mathbf{seed}| = 32$ B, $|\mathbf{salt}| = 16$ B, and $|\mathbf{K}| = 32$ B, respectively. These differences have a negligible impact on the bandwidth comparison, as the public key and ciphertext sizes dominate. The resulting sizes are given in bytes in Table 3.

5 Basic Decoding Failure Rate Analysis

In this section, we provide a basic DFR analysis **Model 1** for HARE under the standard independent Bernoulli distribution assumption, following the methodology of HQC. This serves to establish the fundamental decoding performance before we introduce a more refined error model. Furthermore, we employ certain relaxations in the subsequent derivations such that the estimated DFR serves as an upper bound on the actual DFR. For the sake of simplicity, we utilize equalities rather than inequalities throughout the derivation and omit repetitive references to upper and lower bounds.

Instance	Classical Security	Quantum Security	$ \mathbf{ek}_{\text{KEM}} $	$ \mathbf{dk}_{\text{KEM}} $	$ \mathbf{c}_{\text{KEM}} $	$ \mathbf{K} $	bandwidth
HARE-1	128	64	2025	2073	3608	16	5633
HQC-1			2241	2321	4433	32	6674
HARE-2	128	80	2629	2677	4688	16	7317
HARE-3	192	96	4106	4178	7386	24	11492
HQC-3			4514	4602	8978	32	13492
HARE-5	256	128	6580	6676	11790	32	18370
HQC-5			7237	7333	14421		21658
HARE-7	384	192	13157	13301	23693	48	36850
HARE-9	512	256	21812	22004	39294	64	61106

Table 3: The keypair $(\mathbf{ek}_{\text{KEM}}, \mathbf{dk}_{\text{KEM}})$, ciphertext $\mathbf{c}_{\text{KEM}}$, shared key $\mathbf{K}$ sizes and bandwidth of HARE-KEM and HQC-KEM. All sizes are expressed in bytes.

5.1 Analysis of Error Distribution

We first calculate the probability distribution of each coordinate e'_k of the error vector

$$\mathbf{e}' = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} + \tilde{\mathbf{e}} = (e'_0, \dots, e'_{n-1}),$$

where $\tilde{\mathbf{e}} = (\mathbf{v}_1 - \tilde{\mathbf{m}}\mathbf{G}_2) \bmod^{n-l_1}$ is the error generated by compressing $\mathbf{v}_1$ with covering code.

Based on the error distribution of HQC established in [Theorem 1](#) (see [\[3\]](#) for the proof), we propose the error distribution of HARE in [Theorem 2](#).

Theorem 1. Let $\mathbf{e}^{\text{HQC}} = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}$, the coordinate has distribution

$$\Pr[e_k^{\text{HQC}} = 1] = 2\tilde{p}(1 - \tilde{p})(1 - \frac{w_e}{n}) + ((1 - \tilde{p})^2 + \tilde{p}^2)\frac{w_e}{n},$$

where

$$\tilde{p} = \frac{1}{\binom{n}{w} \binom{n}{w_r}} \sum_{\substack{1 \leq l \leq \min(w, w_r) \\ l \text{ odd}}} \binom{n}{l} \binom{n-l}{w-l} \binom{n-w}{w_r-l}.$$

Theorem 2. Let $p_{\text{HQC}} = \Pr[e_k^{\text{HQC}} = 1]$ in [Theorem 1](#) and $p_{\text{cover}} = \frac{l_2 \cdot R_2}{n \cdot n_2}$ be the error probability generated by the covering code. Then the error distribution is $\Pr[\mathbf{e}'_i = 1] = p_{\text{HQC}}(1 - p_{\text{cover}}) + (1 - p_{\text{HQC}})p_{\text{cover}}$.

According to [\[8\]](#), modeling the dependence between coordinates provides a more accurate estimate of the upper bound on the DFR. In our work, for the sake of simplifying the analysis of erasure decoding, we follow the original HQC model and ignore coordinate dependencies in the subsequent analysis.

5.2 DFR of Reed-Muller Codes

We analyze the upper bound on the probability of a decoding error or an erasure in [Theorem 1](#).

Theorem 1. Let p be the transition probability of the binary symmetric channel and α be the decoding threshold of duplicated RM $[n, k, d]$. With treating some erasures as errors, we can upper bound P_{error} and P_{erasure} as

$$P_{\text{error}} = \sum_{w=0}^n \min \left[\sum_{2j-d > \alpha} 255 \binom{d}{j} \binom{n-d}{w-j}, \binom{n}{w} \right] p^w (1-p)^{n-w}$$

and

$$P_{\text{erasure}} = \sum_{w=0}^n \min \left[\sum_{-\alpha \leq d-2j \leq \alpha} 255 \binom{d}{j} \binom{n-d}{w-j}, \binom{n}{w} \right] p^w (1-p)^{n-w}$$

Proof. Assume the origin codeword without noise is $\mathbf{c}_0 = \mathbf{0}$ and the noise vector is $\mathbf{x} = \mathbf{c}_0 + \mathbf{e}$. During the decoding procedure of the RM code $\mathcal{C}$, the nearest two codewords of $\mathbf{x}$ are $\mathbf{c}_1$ and $\mathbf{c}_2$. Denote $d_0 = w = w_H(\mathbf{c}_0 - \mathbf{x})$, $d_1 = w_H(\mathbf{c}_1 - \mathbf{x})$ and $d_2 = w_H(\mathbf{c}_2 - \mathbf{x})$, then $d_2 \geq d_1$.

- If $\mathbf{c}_1 = \mathbf{c}_0$ and $d_2 - d_1 \leq \alpha$, the output of decoding procedure is erasure. Let $A_{\mathbf{c}_2}^w$ be the event where $\mathbf{c}_1 = \mathbf{c}_0$ and $d_2 - d_1 \leq \alpha$ with fixed w and nonzero codeword $\mathbf{c}_2$. Let $d' = w_H(\mathbf{c}_2 - \mathbf{c}_0)$, then

$$P(A_{\mathbf{c}_2}^w) = \sum_{0 \leq d' - 2j \leq \alpha} \binom{d'}{j} \binom{n - d'}{w - j} p^w (1 - p)^{n - w}.$$

By the union bound applied on all nonzero codewords $\mathbf{c}_2 \in \mathcal{C}$, we obtain the probability upper bounded by:

$$\sum_{w=0}^n \sum_{\mathbf{c}_2 \in \mathcal{C}, \mathbf{c}_2 \neq \mathbf{0}} \sum_{0 \leq d' - 2j \leq \alpha} \binom{d'}{j} \binom{n - d'}{w - j} p^w (1 - p)^{n - w}.$$

There are 255 nonzero codewords in a $[128, 8, 64]$ Reed-Muller code, 254 of weight 64 and one of weight 128. The contribution of the weight 128 vector is smaller than the weight 64 vectors, and we can obtain the same conclusion in duplicated RM code. As a result, the upper bound is

$$\sum_{w=0}^n \sum_{0 \leq d - 2j \leq \alpha} 255 \binom{d}{j} \binom{n - d}{w - j} p^w (1 - p)^{n - w}.$$

- If $\mathbf{c}_1 = \mathbf{c}_0$ and $d_2 - d_1 > \alpha$, the output of decoding procedure is correct.
- If $\mathbf{c}_1 \neq \mathbf{c}_0$ and $w - d_1 \leq \alpha$, we have $d_2 - d_1 \leq w - d_1 \leq \alpha$, the output is erasure. Similar to the first circumstance, we change the range of sum and calculate the upper bound is

$$\sum_{w=0}^n \sum_{-\alpha \leq d - 2j < 0} 255 \binom{d}{j} \binom{n - d}{w - j} p^w (1 - p)^{n - w}.$$

- If $\mathbf{c}_1 \neq \mathbf{c}_0$ and $w - d_1 > \alpha$, the output is either error or erasure. For the sake of simplicity, we treat the output as error. Since the **error** has worse effect on RS code decoding than **erasure**, the decoding ability is underestimated. Similarly, we estimate the upper bound as:

$$\sum_{w=0}^n \sum_{2j - d > \alpha} 255 \binom{d}{j} \binom{n - d}{w - j} p^w (1 - p)^{n - w}.$$

Since the total number of weight- w error patterns is $\binom{n}{w}$, the inner summation cannot exceed this quantity. Thus, we take the minimum between the inner summation and $\binom{n}{w}$ in all cases. This completes the proof. $\square$

5.3 DFR of Shortened Reed-Solomon Codes

Let g be the number of errors and h be the number of erasures, the decoding succeeds as long as $2g + h \leq n_e - k_e$. Thus we have the estimate of DFR in [Theorem 2](#).

Theorem 2. Let p_1 be the probability of a position having error, p_2 be the probability of a position with erasure. Then the DFR of the $RS[n_e, k_e, d_e]$ is,

$$\text{DFR} = \sum_{2g+h > n_e - k_e} \binom{n_e}{g} \binom{n_e - g}{h} p_1^g p_2^h (1 - p_1 - p_2)^{n_e - g - h}$$

Proof. The RS code could correct all vectors with $2g + h \leq n_e - k_e$ where g is the number of errors and h is the number of erasures. Thus, decoding failure occurs if and only if $2g + h > n_e - k_e$. Let p_1 the probability of error and p_2 be the probability of erasure, the probability of having g errors and h erasures is $\binom{n_e}{g} \binom{n_e - g}{h} p_1^g p_2^h (1 - p_1 - p_2)^{n_e - g - h}$. Finally, the DFR sums all the terms when $2g + h > n_e - k_e$. $\square$

6 Refined Decoding Failure Rate Analysis

In [8,14], the authors propose a more accurate model for error distribution and analyze a tighter bound of DFR under the model. Following their approach, we introduce **Model 2**, which adopts a dependent hypergeometric distribution instead of the independent Bernoulli model used in [3].

6.1 Error Distribution Model

In HARE, the total error is

$$\mathbf{e}' = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} + \tilde{\mathbf{e}}.$$

For simplicity, let $w_0, w_1, w_2, w_3 = w_H(\mathbf{x}), w_H(\mathbf{r}_1), w_H(\mathbf{e}), w_H(\tilde{\mathbf{e}})$.

Theorem 3. Let $\mathcal{S}_w^n = \{\mathbf{x} \in \mathbb{F}_2^n \mid w_H(\mathbf{x}) = w\}$. For $\mathbf{x} \xleftarrow{\$} \mathcal{S}_{w_0}^n$ and $\mathbf{y} \xleftarrow{\$} \mathcal{S}_{w_1}^n$, let $p_{w_0, w_1, w} = \Pr[w_H(\mathbf{x} \cdot \mathbf{y}) = w]$, which can be calculated through [8]. Assume all vectors are in $\mathbb{F}_2^n$, we have

$$\Pr[w_H(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{y} \cdot \mathbf{r}_1) = w] = \sum_{l,s} \frac{\binom{s}{l} \binom{n-s}{w+l-s}}{\binom{n}{w+2l-s}} p_{w_0, w_1, s} \cdot p_{w_0, w_1, w+2l-s},$$

and

$$\Pr[w_H(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{y} \cdot \mathbf{r}_1 + \mathbf{e}) = w] = \sum_l \frac{\binom{w_2}{l} \binom{n-w_2}{w+l-w_2}}{\binom{n}{w+2l-w_2}} \Pr[w_H(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{y} \cdot \mathbf{r}_1) = w + 2l - w_2].$$

Thus,

$$\Pr[w_H(\mathbf{e}') = w] = \sum_l \frac{\binom{w_3}{l} \binom{n-w_3}{w+l-w_3}}{\binom{n}{w+2l-w_3}} \Pr[w_H(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{y} \cdot \mathbf{r}_1 + \mathbf{e}) = w + 2l - w_3],$$

Proof. The first identity is derived from the weight distribution of the sum of two independent vectors with hypergeometric distribution. For any two vectors with weights s and $t = w + 2l - s$, the Hamming weight of their sum is w if and only if they share exactly l nonzero positions.

The second and third identities follow the same principle by iteratively accounting for the addition of vectors with fixed Hamming weights w_2 and w_3 . Although the error vector $\mathbf{e}'$ in the KR code may exhibit a smaller Hamming weight in practice, we assume the maximum possible error weight to obtain a conservative estimation of the DFR. $\square$

Theorem 4. Taking truncation into account, we have

$$\Pr[w_H(\text{Truncate}(\mathbf{e}', l_1)) = w] = \sum_{l=w}^{w+n-l_1} \frac{\binom{n_1}{w} \binom{n-n_1}{l-w}}{\binom{n}{l}} \cdot \Pr[w_H(\mathbf{e}') = w].$$

Proof. For a vector $\mathbf{e}'$ with total weight l , the probability that its first l_1 bits contain exactly w nonzero entries is given by the ratio of valid configurations $\binom{l_1}{w} \binom{n-l_1}{l-w}$ to the total possible configurations $\binom{n}{l}$. Summing over all possible values of l yields the result. $\square$

6.2 DFR of RM-RS codes

Theorem 5. Assume that w errors occur in the duplicated RM $[n, k, d]$ code and let α be the decoding threshold. The number of decoding errors $N_{\text{error}, w}$ and erasures $N_{\text{erasure}, w}$ can be upper bounded by

$$N_{\text{error}, w} = \min \left[\sum_{2j-d > \alpha} 255 \binom{d}{j} \binom{n-d}{w-j}, \binom{n}{w} \right],$$

and

$$N_{\text{erasure}, w} = \min \left[\sum_{-\alpha \leq d-2j \leq \alpha} 255 \binom{d}{j} \binom{n-d}{w-j}, \binom{n}{w} - N_{\text{error}, w} \right].$$

The lower bound on the number of correct cases is given by

$$N_{\text{correct}, w} = \max \left[\binom{n}{w} - N_{\text{error}, w} - N_{\text{erasure}, w}, 0 \right].$$

Proof. The calculation of $N_{\text{error},w}, N_{\text{erasure},w}, N_{\text{correct},w}$ is similar with the estimation in [Theorem 1](#). $\square$

Theorem 6. In $RS[n_e, k_e, d_e]$ codes, with g errors and h erasures in the output of RM codes, a decryption failure occurs if and only if $2g + h > n_e - k_e$. Assuming a total of w errors are distributed across all binary positions, we define the upper bound on the number of failure cases, denoted by Γ_w , as

$$\Gamma_w = \sum_{2g+h > n_e - k_e} \binom{n_e}{g} \binom{n_e - g}{h} \cdot \Omega(g, h, w),$$

where

$$\Omega(g, h, w) = \sum_{w_1 + \dots + w_{n_e} = w} \left(\prod_{i=1}^g N_{\text{error}, w_i} \cdot \prod_{i=g+1}^{g+h} N_{\text{erasure}, w_i} \cdot \prod_{i=g+h+1}^{n_e} N_{\text{correct}, w_i} \right).$$

Proof. The proof follows the same logic as that of [Theorem 2](#), with the only difference being the substitution of probability calculations with the corresponding counts of error, erasure, and correct cases. $\square$

Finally, combing the results of [Theorem 4](#) and [Theorem 6](#), we estimate the upper bound of DFR in [Theorem 7](#).

Theorem 7. The total DFR is upper bounded by

$$\text{DFR} = \sum_{w=0}^n \Pr[w_H(\text{Truncate}(\mathbf{e}', l_1)) = w] \cdot \frac{\Gamma_w}{\binom{n}{w}}$$

7 Security Analysis

7.1 Unbalanced-Quasi-Cyclic Syndrome Decoding Problem

In this subsection, we arise the concept of unbalanced-quasi-cyclic syndrome decoding problem (UQCSD) and reduce it to the traditional quasi-cyclic syndrome decoding (QCSD) problem. For the sake of simplicity, we denote this problem in solving version while the decision version is easy to adapt.

Definition 9 (s-QCSD Problem). Given a matrix $\mathbf{H} \in \mathbb{F}_2^{(sn-n) \times sn}$ and a vector $\mathbf{s} \in \mathbb{F}_2^{sn-n}$, the goal is to find a vector $\mathbf{e} \in \mathbb{F}_2^{sn}$ satisfying $\mathbf{H}\mathbf{e} = \mathbf{s}$. Let $\mathbf{e} = (\mathbf{e}_0, \dots, \mathbf{e}_{s-1})$ where $\mathbf{e}_i \in \mathbb{F}_2^n$ and $w_H(\mathbf{e}_i) = w$ for all $0 \leq i < s$. We call this problem as s-QCSD(n, w) problem.

Definition 10 (UQCSD Problem). Given a matrix $\mathbf{H} \in \mathbb{F}_2^{2n \times 3n}$ and a vector $\mathbf{s} \in \mathbb{F}_2^{2n}$, the goal is to find a vector $\mathbf{e} \in \mathbb{F}_2^{3n}$ satisfying $\mathbf{H}\mathbf{e} = \mathbf{s}$. Let $\mathbf{e} = (\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2)$ where $\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2 \in \mathbb{F}_2^n \times \mathbb{F}_2^n \times \mathbb{F}_2^n$, we ask $w_H(\mathbf{e}_0) = w_H(\mathbf{e}_2) = w_0$ and $w_H(\mathbf{e}_1) = w_1$. We call this problem as UQCSD(n, w_0, w_1) problem.

If $w_0 = w_1$, it is obvious that the UQCSD problem is equivalent with 3-QCSD problem. Otherwise, we make a reduction in [Theorem 8](#).

Theorem 8. If an adversary is able to solve UQCSD(n, w_0, w_1) with advantage ε , then we can construct an adversary solving a 3-QCSD($n, (2w_0 + w_1)/3$) problem with advantage ε' , where $\varepsilon' = \frac{\binom{n}{w_0}^2 \binom{n}{w_1}}{\binom{3n}{2w_0 + w_1}} \varepsilon$.

Proof. Let $\mathcal{A}$ be the adversary for UQCSD(n, w_0, w_1), we then construct $\mathcal{A}'$ for a 3-QCSD problem. We denote $w = (2w_0 + w_1)/3$ and assume w is an integer without loss of generality. Given an instance $(\mathbf{H}, \mathbf{s})$ of a 3-QCSD(n, w) problem, we then generate a random permutation matrix $\mathbf{P} \in \mathbb{F}_2^{3n \times 3n}$. The adversary $\mathcal{A}'$ call the oracle of $\mathcal{A}$ to solve the UQCSD(n, w_0, w_1) with the input of $(\mathbf{H}\mathbf{P}, \mathbf{s})$. Let $\mathbf{e}$ be the answer of the 3-QCSD problem and $(\mathbf{e}'_0, \mathbf{e}'_1, \mathbf{e}'_2) \leftarrow \mathbf{P}\mathbf{e}$. Let $\mathcal{B}$ be the event

of $w_H(\mathbf{e}'_0) = w_H(\mathbf{e}'_2) = w_0$ and $w_H(\mathbf{e}'_1) = w_1$. When $\mathbf{B}$ occurs, the instance $(\mathbf{HP}, \mathbf{s})$ is a standard UQCSD problem thus $\mathcal{A}$ could solve it with advantage ε .

Now we only need to calculate the probability of event $\mathbf{B}$. Since $\mathbf{P}$ is a random permutation matrix, we have $\Pr(\mathbf{B}) = \frac{\binom{n}{w_0}^2 \binom{n}{w_1}}{\binom{3n}{2w_0+w_1}}$. As a result, the advantage of $\mathcal{A}'$ is $\varepsilon' = \frac{\binom{n}{w_0}^2 \binom{n}{w_1}}{\binom{3n}{2w_0+w_1}} \varepsilon$. $\square$

7.2 Security Assumption

The security of our scheme relies on the 2-DQCSD-P problem and 3-DUQCSD-PT problem. These problems are variants of QCSD and UQCSD problems, while the introduced conditions of parity and truncation aim at avoiding trivial distinguishers.

Let $\mathbb{F}_{2,b_1}^n = \{\mathbf{h} \in \mathbb{F}_2^n \mid w_H(\mathbf{h}) = b_1 \bmod 2\}$, i.e., all binary vectors with length n and parity b_1 . Let

$$\mathbb{F}_{2,b_1}^{n \times 2n} = \{\mathbf{H} = (\mathbf{I}_n, \text{rot}(\mathbf{h})) \in \mathbb{F}_2^{n \times 2n} \mid \mathbf{h} \in \mathbb{F}_{2,b_1}^n\}, \text{ and}$$

$$\mathbb{F}_{2,b_1,b_2}^{2n \times 3n} = \left\{ \mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}_1) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{h}_2) \end{pmatrix} \in \mathbb{F}_2^{2n \times 3n} \mid \mathbf{h}_1 \in \mathbb{F}_{2,b_1}^n, \mathbf{h}_2 \in \mathbb{F}_{2,b_2}^n \right\}.$$

Definition 11 (2-QCSD-P Distribution). Let n, w, b_1 be positive integers and let $b_2 = w + b_1 \times w \bmod 2$. The 2-Quasi-Cyclic Syndrome Decoding with Parity Distribution 2-QCSD-P(n, w, b_1, b_2) samples $\mathbf{H} \in \mathbb{F}_{2,b_1}^{n \times 2n}$ and $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2) \xleftarrow{\$} \mathbb{F}_2^{2n}$ such that $w_H(\mathbf{e}_1) = w_H(\mathbf{e}_2) = w$, computes $\mathbf{s}^\top = \mathbf{H}\mathbf{e}^\top$ and outputs $(\mathbf{H}, \mathbf{s}) \in \mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$.

Definition 12 (2-DQCSD-P Problem). Let n, w, b_1 be positive integers and let $b_2 = w + b_1 \times w \bmod 2$. Given $(\mathbf{H}, \mathbf{s}) \in \mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$, the Decisional 2-Quasi-Cyclic Syndrome Decoding with Parity Problem 2-DQCSD-P(n, w, b_1, b_2) asks to decide with non-negligible advantage whether the pair $(\mathbf{H}, \mathbf{s})$ came from the 2-QCSD-P(n, w, b_1, b_2) distribution or uniform distribution over $\mathbb{F}_{2,b_1}^{n \times 2n} \times \mathbb{F}_{2,b_2}^n$.

Definition 13 (3-UQCSD-PT Distribution). Let n, w_0, w_1, b_1, b_2, l be positive integers and let $b_3 = w_0 + b_1 \times w_0 \bmod 2$. Specifically, we define that 3-Unbalanced-Quasi-Cyclic Syndrome Decoding with Parity and Truncation Distribution 3-UQCSD-PT($n, w_0, w_1, b_1, b_2, b_3, l$) samples $\mathbf{H} \xleftarrow{\$} \mathbb{F}_{2,b_1,b_2}^{2n \times 3n}$ and $\mathbf{e} = (\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2) \xleftarrow{\$} \mathbb{F}_2^{n \times n}$ such that $w_H(\mathbf{e}_0) = w_H(\mathbf{e}_2) = w_0$ and $w_H(\mathbf{e}_1) = w_1$, computes $\mathbf{s}^\top = \mathbf{H}\mathbf{e}^\top$ where $\mathbf{s} = (\mathbf{s}_0, \mathbf{s}_1)$ and outputs $(\mathbf{H}, (\mathbf{s}_0, \text{Truncate}(\mathbf{s}_1, l))) \in \mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_2^l)$.

Definition 14 (3-DUQCSD-PT Problem). Let n, w_0, w_1, b_1, b_2, l be positive integers and let $b_3 = w_0 + b_1 \times w_0 \bmod 2$. Given $(\mathbf{H}, (\mathbf{s}_0, \mathbf{s}_1)) \in \mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_2^l)$, the Decisional 3-Unbalanced-Quasi-Cyclic Syndrome Decoding with Parity and Truncation Problem 3-DUQCSD-PT($n, w_0, w_1, b_1, b_2, b_3, l$) asks to decide with non-negligible advantage whether $(\mathbf{H}, (\mathbf{s}_0, \mathbf{s}_1))$ comes from 3-UQCSD-PT($n, w_0, w_1, b_1, b_2, b_3, l$) distribution or the uniform distribution over $\mathbb{F}_{2,b_1,b_2}^{2n \times 3n} \times (\mathbb{F}_{2,b_3}^n \times \mathbb{F}_2^l)$.

7.3 Security Proof

7.3.1 IND-CPA Security

In this section, we prove the IND-CPA security of the HARE-PKE scheme relying the security assumptions in Section 7.2. Let $b_1 = \mathbf{h}(1) \bmod 2$, $b_2 = w + b_1 \times w \bmod 2$ and $b_3 = w_r + b_1 \times w_r \bmod 2$. Let $l_1 = n_1 \times n_2$.

Theorem 9. For any IND-CPA adversary $\mathcal{A}$ against the HARE-PKE scheme, there exist adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:

$$\text{Adv}_{\text{HARE-PKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_1) + \text{Adv}_{3\text{-DUQCSD-PT}}(\mathcal{B}_2). \quad (5)$$

Proof. We divide our proof in 4 stages.

(1) **Remove the compression technique.** Let HARE-PKE₀ be our PKE scheme without the code compression, i.e., keeping $\mathbf{v}$ instead of $\tilde{\mathbf{m}}, \mathbf{v}_2$ as the ciphertext. Let Game₀ be the IND-CPA game for HARE-PKE₀ and Game₁ be the IND-CPA game for HARE-PKE. For any adversary $\mathcal{A}_1$ against HARE-PKE, we construct $\mathcal{A}_0$ to break the IND-CPA of HARE-PKE₀. The procedures are as follows.

1. $\mathcal{A}_0$ receives the encryption key $\mathbf{ek}$ from HARE-PKE_0 and sends it to $\mathcal{A}_1$.
2. $\mathcal{A}_0$ gets $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{1,\text{CHOOSE}}(\mathbf{ek})$ and sends it to the challenger of HARE-PKE_0 .
3. The challenger selects $b \xleftarrow{\$} \{0, 1\}$, $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$ and generates $\mathbf{c} \leftarrow \text{HARE-PKE}_0.\text{Encrypt}(\mathbf{ek}, \mathbf{m}_b, \theta)$.
4. Let $(\mathbf{u}, \mathbf{v}) \leftarrow \mathbf{c}$, $\mathcal{A}_0$ calculates $\mathbf{v}_1 = \mathbf{v}[0, l_2 - 1]$, $\mathbf{v}_2 = \mathbf{v}[l_2, l_1]$, and $\tilde{\mathbf{m}} = \mathcal{C}_2.\text{Decode}(\mathbf{v}_1)$. Then $\mathcal{A}_0$ sends the challenge ciphertext $(\mathbf{u}, \tilde{\mathbf{m}}, \mathbf{v}_2)$ to $\mathcal{A}_1$.
5. $\mathcal{A}_0$ outputs the guess bit b' received by $\mathcal{A}_1$.

The transformation from $(\mathbf{u}, \mathbf{v})$ to $(\mathbf{u}, \tilde{\mathbf{m}}, \mathbf{v}_2)$ is a publicly computable mapping requiring no secret information. Since $\mathcal{A}_0$ applies the exact compression defined in HARE-PKE , the challenge ciphertext provided to $\mathcal{A}_1$ is identically distributed to a real ciphertext in HARE-PKE . Thus, $\mathcal{A}_1$ has a perfectly identical view in the reduction as in the real IND-CPA game, which directly implies:

$$\text{Adv}_{\text{HARE-PKE}}^{\text{IND-CPA}}(\mathcal{A}_1) = \text{Adv}_{\text{HARE-PKE}_0}^{\text{IND-CPA}}(\mathcal{A}_0). \quad (6)$$

(2) Replace the encryption key. We first define the game against HARE-PKE_0 . For the sake of simplicity, we use the vector $\mathbf{h}$ as part of the encryption key instead of the seed to generate it.

Game₁(λ)

1. $\text{seed}_{\text{PKE}} \xleftarrow{\$} \mathcal{B}^{|\text{seed}_{\text{PKE}}|}$
2. $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow \text{HARE-PKE}_0.\text{Keygen}(\text{seed}_{\text{PKE}})$
3. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\mathbf{ek}_{\text{PKE}})$
4. $b \xleftarrow{\$} \{0, 1\}$, $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$
5. $\mathbf{c}_{\text{PKE}} \leftarrow \text{HARE-PKE}_0.\text{Encrypt}(\mathbf{ek}_{\text{PKE}}, \mathbf{m}_b, \theta)$
6. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\mathbf{ek}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. **return** $(b = b')$

Then we forget the decryption key $\mathbf{dk}_{\text{PKE}}$, replace $\mathbf{s}$ of the encryption key with a random vector with parity b_2 , and construct **Game₂** with other steps unchanged.

Game₂(λ)

1. $\text{seed}_{\text{PKE}} \xleftarrow{\$} \mathcal{B}^{|\text{seed}_{\text{PKE}}|}$
2. $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow \text{HARE-PKE}_0.\text{Keygen}(\text{seed}_{\text{PKE}})$
3. $\mathbf{s} \xleftarrow{\$} \mathbb{F}_{2, b_2}^n$
4. $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow ((\text{seed}_{\text{PKE.ek}}, \mathbf{s}), \mathbf{0})$
5. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\mathbf{ek}_{\text{PKE}})$
6. $b \xleftarrow{\$} \{0, 1\}$, $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$
7. $\mathbf{c}_{\text{PKE}} \leftarrow \text{HARE-PKE}_0.\text{Encrypt}(\mathbf{ek}_{\text{PKE}}, \mathbf{m}_b, \theta)$
8. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\mathbf{ek}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
9. **return** $(b = b')$

Suppose there exists an adversary $\mathcal{B}_1$ which is able to distinguish **Game₁** and **Game₂** with advantage ε under security parameter λ . Then we construct an adversary to solve the 2-DQCSD-P problem with the same advantage.

Adversary against 2-DQCSD-P($\mathbf{H}, \mathbf{s}$)

1. Compute $\mathbf{h}$ from $\mathbf{H} = (\mathbf{I}_n, \text{rot}(\mathbf{h}))$
2. Set $\mathbf{ek}_{\text{PKE}} \leftarrow (\mathbf{h}, \mathbf{s})$
3. Get $\mathbf{m}_0, \mathbf{m}_1 \leftarrow \mathcal{B}_{1,\text{CHOOSE}}(\mathbf{ek}_{\text{PKE}})$
4. Sample $b \xleftarrow{\$} \{0, 1\}$ and $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$
5. Compute $\mathbf{c}_{\text{PKE}} \leftarrow \text{HARE-PKE}_0.\text{Encrypt}(\mathbf{ek}_{\text{PKE}}, \mathbf{m}_b, \theta)$
6. Get $b' \leftarrow \mathcal{B}_{1,\text{GUESS}}(\mathbf{ek}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$
7. If $b' = \text{Game}_1$, output 2-QCSD-P(n, w, b_1, b_2) distribution
8. If $b' = \text{Game}_2$, output $\mathcal{U}(\mathbb{F}_{2, b_1}^{n \times 2n} \times \mathbb{F}_{2, b_2}^n)$ distribution

The only difference between Game_1 and Game_2 is the distribution of $(\mathbf{h}, \mathbf{s})$. Thus, the advantage ε of distinguishing Game_1 and Game_2 is precisely the advantage $\text{Adv}_{2\text{-DQCSD-P}}$ of distinguishing the distribution of $(\mathbf{h}, \mathbf{s})$ from the random distribution.

(3) Replace the ciphertext. Then we construct Game_3 by replacing the ciphertext. Instead of sampling $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$ with correct weight, we pick random vectors with expected parity.

$\text{Game}_3(\lambda)$
1. $\text{seed}_{\text{PKE}} \xleftarrow{\\$} \mathcal{B}^{ \text{seed}_{\text{PKE}} }$ 2. $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow \text{HARE-PKE}_0.\text{Keygen}(\text{seed}_{\text{PKE}})$ 3. $\mathbf{s} \xleftarrow{\\$} \mathbb{F}_{2, b_2}^n$ 4. $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow ((\text{seed}_{\text{PKE}}, \mathbf{ek}), \mathbf{s}, \mathbf{0})$ 5. $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}_{\text{CHOOSE}}(\mathbf{ek}_{\text{PKE}})$ 6. $b \xleftarrow{\\$} \{0, 1\}$ 7. $\mathbf{e} \xleftarrow{\\$} \mathbb{F}_2^n, (\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\\$} \mathbb{F}_{2, w_r}^n \times \mathbb{F}_{2, w_r}^n$ 8. $\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$ 9. $\mathbf{v} \leftarrow \mathbf{m}_b \mathbf{G}_1 + \text{Truncate}(\mathbf{s}\mathbf{r}_2 + \mathbf{e}, l_1)$ 10. $\mathbf{c}_{\text{PKE}} \leftarrow (\mathbf{u}, \mathbf{v})$ 11. $b' \leftarrow \mathcal{A}_{\text{GUESS}}(\mathbf{ek}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$ 12. return $(b = b')$

Suppose there exists an adversary $\mathcal{B}_2$ which is able to distinguish Game_2 and Game_3 with advantage ε under security parameter λ . Then we construct an adversary to solve the 3-DUQCSD-PT problem with the same advantage.

Adversary against 3-DUQCSD-PT($\mathbf{H}, (\mathbf{u}, \mathbf{v})$)
1. Compute $\mathbf{h}$ and $\mathbf{s}$ from $\mathbf{H} = \begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}$ 2. Compute $\mathbf{ek}_{\text{PKE}} \leftarrow (\mathbf{h}, \mathbf{s})$ 3. Get $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{B}_{2, \text{CHOOSE}}(\mathbf{ek}_{\text{PKE}})$ 4. Sample $b \xleftarrow{\\$} \{0, 1\}$ 5. Compute $\mathbf{c}_{\text{PKE}} \leftarrow (\mathbf{u}, \mathbf{m}_b \mathbf{G}_1 + \mathbf{v})$ 6. Get $b' \leftarrow \mathcal{B}_{2, \text{GUESS}}(\mathbf{ek}_{\text{PKE}}, \mathbf{c}_{\text{PKE}})$ 7. If $b' = \text{Game}_2$, output 3-UQCSD-PT($n, w_0, w_1, b_1, b_2, b_3, l$) distribution 8. If $b' = \text{Game}_3$, output $\mathcal{U}(\mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_2^{n-\ell}))$ distribution

With

$$(\mathbf{u}, \mathbf{v} - \mathbf{m}_b \mathbf{G}_1)^\top = ((\mathbf{I}_n \ \mathbf{0} \ \text{rot}(\mathbf{h})) \cdot (\mathbf{r}_1 \ \mathbf{e} \ \mathbf{r}_2)^\top, \text{Truncate}((\mathbf{0} \ \mathbf{I}_n \ \text{rot}(\mathbf{s})) \cdot (\mathbf{r}_1 \ \mathbf{e} \ \mathbf{r}_2)^\top, \ell)),$$

the only difference between Game_2 and Game_3 lies in the pair

$$\left(\begin{pmatrix} \mathbf{I}_n & \mathbf{0} & \text{rot}(\mathbf{h}) \\ \mathbf{0} & \mathbf{I}_n & \text{rot}(\mathbf{s}) \end{pmatrix}, (\mathbf{u}, \mathbf{v} - \mathbf{m}_b \mathbf{G}_1) \right)$$

follows the 3-DUQCSD-PT($n, w_0, w_1, b_1, b_2, b_3, l$) distribution or a uniform distribution with parity over $\mathbb{F}_{2, b_1, b_2}^{2n \times 3n} \times (\mathbb{F}_{2, b_3}^n \times \mathbb{F}_2^l)$. As a result, the advantage of solving 3-DUQCSD-PT is the same as the advantage of $\mathcal{B}_2$.

(4) Conclusion. In Game_3 , since $\mathbf{u}$ is computed independently of the challenge bit b and $\mathbf{v}$ is masked by a random vector $\mathbf{e}$, the advantage on Game_3 is 0.

With the previous proof chain, we have

$$\begin{aligned}
\text{Adv}_{\text{HARE-PKE}}^{\text{IND-CPA}}(\mathcal{A}) &= \text{Adv}_{\text{HARE-PKE}_0}^{\text{IND-CPA, Game}_1}(\mathcal{A}) \\
&\leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_1) + \text{Adv}_{\text{HARE-PKE}_0}^{\text{IND-CPA, Game}_2}(\mathcal{A}) \\
&\leq \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_1) + \text{Adv}_{3\text{-DUQCSD-PT}}(\mathcal{B}_2) + \text{Adv}_{\text{HARE-PKE}_0}^{\text{IND-CPA, Game}_3}(\mathcal{A}) \\
&= \text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_1) + \text{Adv}_{3\text{-DUQCSD-PT}}(\mathcal{B}_2).
\end{aligned}$$

which completes the proof. $\square$

7.3.2 IND-CCA2 Security

By constructing HARE-KEM with the salted Fujisaki-Okamoto transform [12,13,15], we prove the IND-CCA2 security of HARE-KEM.

Considering the DFR of schemes, we give the following definitions which guarantee δ -correct.

Definition 15 (δ -correct PKE [13]). A PKE is δ -correct if:

$$\mathbb{E} \left(\max_{\mathbf{m} \in \mathcal{M}} \Pr [\text{PKE.Decrypt}(\mathbf{dk}_{\text{PKE}}, \mathbf{c}_{\text{PKE}}) \neq \mathbf{m} \mid \mathbf{c}_{\text{PKE}} \leftarrow \text{PKE.Encrypt}(\mathbf{ek}_{\text{PKE}}, \mathbf{m})] \right) \leq \delta. \quad (9)$$

where the expectation is taken over $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow \text{PKE.Keygen}()$.

Definition 16 (δ -correct KEM [13]). A KEM is δ -correct if:

$$\Pr \left[\text{KEM.Decaps}(\mathbf{dk}_{\text{KEM}}, \mathbf{c}_{\text{KEM}}) \neq \mathbf{K} \mid \begin{array}{l} (\mathbf{ek}_{\text{KEM}}, \mathbf{dk}_{\text{KEM}}) \leftarrow \text{KEM.Keygen}(); \\ (\mathbf{K}, \mathbf{c}_{\text{KEM}}) \leftarrow \text{KEM.Encaps}(\mathbf{ek}_{\text{KEM}}) \end{array} \right] \leq \delta. \quad (7)$$

In HARE-PKE and HARE-KEM, the δ is upper bounded by the analysis of DFR in Section 5. Thus, we can upper bound the advantage of HARE-KEM in Theorem 10 by combining the Theorem 13 from [15] and the result in Theorem 9.

Theorem 10. If HARE-PKE is δ -correct, for any IND-CCA2 adversary $\mathcal{A}$ against the HARE-KEM scheme issuing at most q_{RO} queries to $\mathbf{G}$ and q_D queries to the HARE-KEM.Decaps oracle, there exists adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:

$$\begin{aligned}
\text{Adv}_{\text{HARE-KEM}}^{\text{IND-CCA2}}(\mathcal{A}) &\leq \frac{1}{2^{|\mathbf{K}|} \cdot 2^{|\text{salt}|}} + \frac{3q_{RO}}{2^{|\mathbf{K}|}} + (q_{RO} + q_D) \cdot \delta \\
&\quad + 2 \cdot (\text{Adv}_{2\text{-DQCSD-P}}(\mathcal{B}_1) + \text{Adv}_{3\text{-DUQCSD-PT}}(\mathcal{B}_2)).
\end{aligned} \quad (13)$$

7.3.3 Known Attacks

We categorize the potential attacks into two main categories. First, **structural attacks** exploit the cyclic properties of quasi-cyclic codes when $X^n - 1$ possesses low-degree factors [16,17]. To mitigate these threats, we select n as a primitive prime, ensuring that the polynomial $X^n - 1$ has only two irreducible factors. Consequently, such structural attacks cannot be efficiently applied to our scheme.

Second, we consider attacks targeting the 2-DQCSD-P and 3-DUQCSD-PT problems. The most efficient methods for solving the Syndrome Decoding (SD) problem are Information Set Decoding (ISD) algorithms, such as those presented in [18,19,20,21]. These algorithms remain applicable to 2-DQCSD-P and 3-DUQCSD-PT. In the quasi-cyclic setting, a speed-up via the Decoding One Out of Many (DOOM) technique [22] should be accounted for, where the complexity gain is $O(\sqrt{n})$ for both 2-DQCSD-P and 3-DUQCSD-PT. We use the open-source estimator from [23] to evaluate the concrete bit security of these problems, with the results shown in Table 4. The values reported in the table correspond to classical complexity, while the quantum complexity is obtained by halving the security level.

Instance	Problem	Prange [18]	Stern [19]	BJMM [21]	Target Security
HARE-1	3-DUQCSD-PT	154	137	134	128
	2-DQCSD-P	164	145	143	
HARE-2	3-DUQCSD-PT	183	164	161	128
	2-DQCSD-P	191	171	169	
HARE-3	3-DUQCSD-PT	223	204	201	192
	2-DQCSD-P	235	213	211	
HARE-5	3-DUQCSD-PT	287	266	263	256
	2-DQCSD-P	299	275	273	
HARE-7	3-DUQCSD-PT	414	391	387	384
	2-DQCSD-P	425	400	397	
HARE-9	3-DUQCSD-PT	545	520	516	512
	2-DQCSD-P	559	532	529	

Table 4: The ISD bit-security of HARE parameter sets.

Instance	KeyGen	Encaps	Decaps
HARE-2	8258.9	17908.5	26841.0
HARE-5	35579.4	74753.8	114053.8
HARE-7	106450.7	220198.8	330734.3
HARE-9	226932.0	464810.5	702153.9

Table 5: Performances (in kiloCycles) of the reference implementation of HARE.

8 Performance Analysis

8.1 Experimental Setup

Our benchmarks were conducted on a CentOS-based x86_64 system running CentOS Linux 8.2.2004 with kernel version 4.18.0-193.28.1.el8_2.x86_64. The platform was equipped with an Intel Xeon Platinum 8269CY CPU @ 2.50GHz, featuring 2 sockets, 80 logical CPUs (2 threads per core), and operating at an observed frequency of approximately 3500.652 MHz (with a maximum of 3800 MHz). The system provided 187 GiB of memory and supported instruction set extensions including AVX2, PCLMULQDQ, and AVX512. The software toolchain consisted of GCC/G++ version 8.3.1, with the build process managed using CMake version 3.11.4 and the Unix Makefiles generator. The detailed description of the test environment is provided in the attached files.

8.2 Reference Implementation

It should be noted that HARE-2/5/7/9 correspond to the call of Institute of Commercial Cryptography Standards, while HARE-1/3/5 are used for a fair comparison with the original HQC. Therefore, we only implement HARE-2/5/7/9. The performances of our reference implementation on the aforementioned benchmark platform are given in Table 5. The implementation is written in C and compiled with flags `-std=c99 -Wpedantic -Wall -Wextra -O2`.

8.3 Optimized Implementation

The performances of our optimized implementation on the aforementioned benchmark platform are given in Table 6. Our optimized version leverages x86 instructions and is compiled with flags `-O3 -march=x86-64 -mavx2 -mpclmul -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`.

Instance	KeyGen	Encaps	Decaps
HARE-2	230.7	937.6	1506.9
HARE-5	748.7	2702.0	4729.3
HARE-7	1979.5	6468.5	10576.7
HARE-9	4616.2	13074.0	21472.9

Table 6: Performances (in kiloCycles) of the optimized implementation of HARE.

References

1. Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. pages 124–134, 1994.
2. National institute of standards and technology: Post-quantum cryptography project. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2016.
3. Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaleb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and I Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.
4. Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
5. Qian Guo, Thomas Johansson, and Carl Löndahl. Solving LPN using covering codes. pages 1–20, 2014.
6. Markku K Kaikkonen and Petri Rosendahl. New covering codes from an ads-like construction. *IEEE Transactions on Information Theory*, 49(7):1809–1812, 2003.
7. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In *IACR International Conference on Public-Key Cryptography*, pages 37–65. Springer, 2020.
8. Sebastian Bitzer, Jean-Christophe Deneuville, Emma Munisamy, Bharath Puritpli, Stefan Ritterhoff, and Antonia Wachter-Zeh. Hqc beyond the standard: Ciphertext compression and refined dfr analysis. *Cryptology ePrint Archive*, 2025.
9. Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error-correcting codes*, volume 16. Elsevier, 1977.
10. Shu Lin, Daniel J Costello, et al. *Error control coding*, volume 2. Prentice hall Scarborough, 2001.
11. Api specification for public key cryptography. Technical report, Institute of Commercial Cryptography Standards, 2025.
12. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.
13. Lewis Glabush, Kathrin Hövelmanns, and Douglas Stebila. Tight multi-challenge security reductions for key encapsulation mechanisms. *IACR Cryptol. ePrint Arch.*, 2025:343, 2025.
14. Alessandro Annechini, Alessandro Barengi, and Gerardo Pelosi. On the decoding failure rate of hqc. *Cryptology ePrint Archive*, 2026.
15. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In *Theory of Cryptography Conference*, pages 341–371. Springer, 2017.
16. Qian Guo, Thomas Johansson, and Carl Löndahl. A new algorithm for solving ring-lpn with a reducible polynomial. *IEEE Transactions on Information Theory*, 61(11):6204–6212, 2015.
17. Carl Löndahl, Thomas Johansson, Masoumeh Koochak Shooshtari, Mahmoud Ahmadian-Attari, and Mohammad Reza Aref. Squaring attacks on mceliece public-key cryptosystems using quasi-cyclic codes of even dimension. *Designs, Codes and Cryptography*, 80(2):359–377, 2016.
18. Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962.
19. Jacques Stern. A method for finding codewords of small weight. In *Coding Theory and Applications: 3rd International Colloquium Toulon, France, November 2–4, 1988 Proceedings 3*, pages 106–113. Springer, 1989.
20. Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\mathcal{O}(2^{0.054n})$. In *Advances in Cryptology—ASIACRYPT 2011: 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4–8, 2011. Proceedings 17*, pages 107–124. Springer, 2011.
21. Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1+1=0$ improves information set decoding. In *Advances in Cryptology—EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15–19, 2012. Proceedings 31*, pages 520–536. Springer, 2012.
22. Nicolas Sendrier. Decoding one out of many. In *International Workshop on Post-Quantum Cryptography*, pages 51–67. Springer, 2011.
23. Yu Yu, Kang Yang, Xiao Wang, Anyu Wang, Tianrui Wang, Hanlin Liu, Xinpeng Hao, and Juanru Li. Estimator of lpn problems over any finite fields and power-of-two rings for pcg and mpc applications. <https://lpnestimator.com>, 2025.